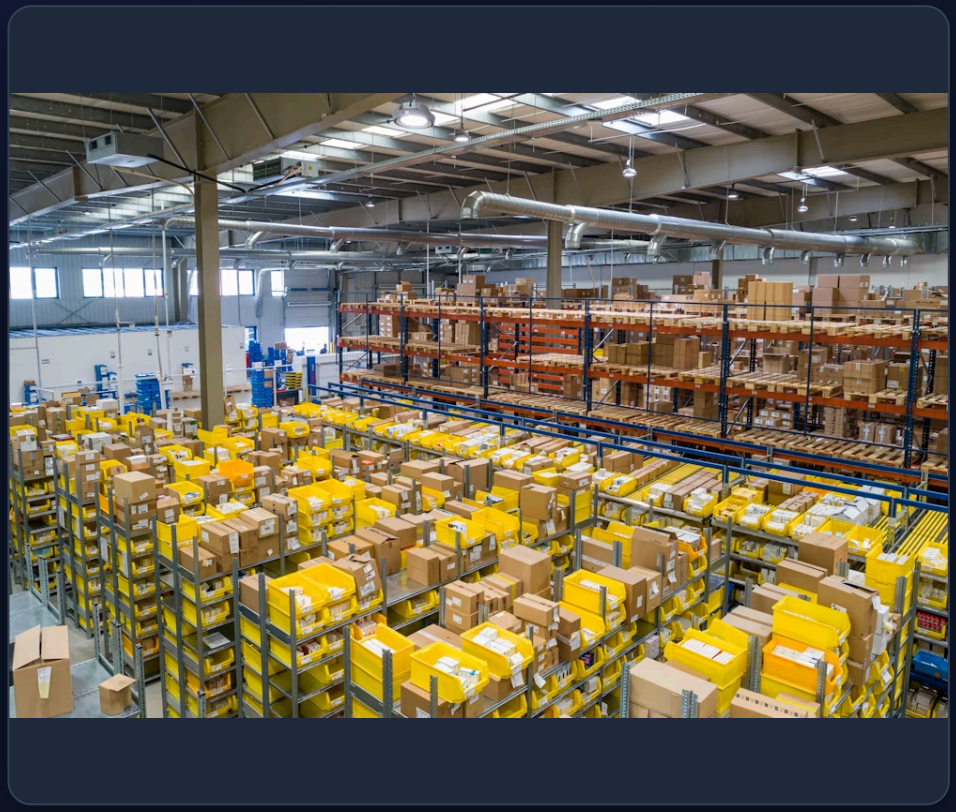
지능형 로보틱스 최종 발표

RGB-D 기반 실내 재고창고 선반 상품 확인 로봇 시스템

팀명: 호로록 | 2271065 한유리

발표 목차

- 01. 프로젝트 개요 및 목표
- 02. 추진 배경 (2D Navigation의 한계)
- 03. 3D 기반 핵심 목표 및 전략
- 04. 시스템 아키텍처 및 데이터 흐름
- 05. 시스템 검증 (시나리오별 테스트)
- 06. 어려웠던 점 및 구현 한계



프로젝트 개요 PROJECT OVERVIEW

프로젝트 정의

RGB-D 기반 실내 재고창고
선반 상품 확인 로봇 시스템

활용 기술

- ROS2
- Gazebo
- RGB-D Camera, RTAB-Map, Nav2

핵심 목표

사용자가 원하는 상품을 입력하면

→ 객체 탐지

→ 상품 3D 위치 추정

→ 최적 관측 위치 계산

→ 자율주행 이동

→ 상품 존재 여부 확인

문제 정의: 2D Navigation의 한계

기존 2D 방식의 한계점

- **선반 높이 정보 부재:** 2D LiDAR 데이터만으로는 상품이 놓인 정확한 수직적 위치를 파악할 수 없어 입체적인 재고 확인이 불가능합니다.
- **상단 장애물 인식 불가:** 로봇 스캔 범위를 벗어난 상단 돌출 구조물을 인식하지 못해 자율 주행 중 충돌 위험이 심각하게 증가합니다.

RGB-D 기반 3D 해결 전략

- **객체 3D 위치 추정:** 카메라의 깊이(Depth) 데이터를 적극 활용하여 대상 상품의 실제 3차원 공간 좌표를 정밀하게 산출합니다.
- **3D Viewpoint Planning:** 주변 물체로 인한 차폐 여부를 고려하여 상품을 가장 명확하게 식별할 수 있는 최적 관측 위치를 생성합니다.

3D 기반 의사결정 시스템의 핵심 목표



객체 3D 위치 추정

YOLO 기반의 2D 객체 인식 영상과 Depth 데이터를 융합하여, 대상 상품의 실제 3차원 위치(거리 및 높이)를 정확하게 추정합니다.



최적 관측 위치 계산

상품의 식별 가능 거리와 주변 장애물로 인한 차폐(Occlusion) 여부를 실시간으로 판단하여, 가장 완벽한 시야각을 확보하는 Viewpoint를 도출합니다.



3D 기반 안전 주행

상단 돌출 장애물 등 2D 센서로 감지하기 어려운 3D 충돌 가능 영역을 사전에 계산하고 제거하여, 로봇의 안전한 자율 주행 경로를 보장합니다.

시스템 아키텍처 (전체 데이터 흐름)

1 RGB-D Camera

입력

RGB/Depth Image, Point Cloud

출력

객체 탐지에 필요한 센서 데이터

2 YOLO Detection

입력

RGB Image (목표 상품 탐지)

출력

Bounding Box, Class Label, Confidence

3 Position Estimation

입력

Bounding Box, Depth Image

출력

Object Pose (X, Y, Z)

선반 내 객체의 실제 3차원 위치/높이 파악

4 Viewpoint Planning

고려사항

카메라 FoV, 선반 차폐, 접근/충돌 가능성

출력

최적의 관측 위치 (Selected Goal Pose)

5 Nav2 Navigation

모듈

Global/Local Planner, Recovery Behavior

입출력

Goal Pose ➔ cmd_vel, Planned Path

6 Verification

성공

목표 도착 후 객체 재검출 및 상품 존재 확인

실패

후보 위치 재탐색 또는 상품 없음 보고

주요 ROS Topic 구성

Sensor

- /camera/camera/image_raw
- /camera/camera/depth/image_raw
- /camera/camera/points
- /scan
- /odom

Perception

- /detections
- /object_pose

Navigation

- /map
- /goal_pose
- /cmd_vel
- /tf
- /tf_static

검증 1. 3D 시야 확보 및 차폐(Occlusion) 극복

객체의 '**3D 높이(Z 좌표)**'를 알아야만 해결할 수 있는 인지 및 관측 위치 계획을 검증합니다.

Case 1

다단 선반 화각 한계 극복

상황 목표 상품이 로봇 카메라보다 훨씬 높은 3층 선반 안쪽에 위치함.

검증 목표 2D 바닥 위치만 보고 선반 바로 앞까지 다가가는 것이 아니라, 카메라 화각(FoV)을 계산하여 선반에서 충분히 뒤로 물러난(이격된) 최적의 관측 위치를 스스로 도출하는지 확인합니다.

Case 2

전면 장애물 차폐 극복

상황 목표 상품 바로 앞에 다른 큰 상자가 놓여 있어, 정면(첫 후보 위치)에서 상품이 가려져 보이지 않음.

검증 목표 확인 실패 시 종료되지 않고, BT(Behavior Tree) 복구 로직이 작동하여 측면의 새로운 뷰포인트로 이동해 상품을 다시 찾아내는지 확인합니다.

Case 3

기본 3D 위치 추정 (베이스라인)

상황 장애물이 없는 1층 선반에 상품이 놓여 있음.

검증 목표 YOLO bounding box와 Depth 이미지가 결합되어, map 좌표계 기준으로 객체의 정확한 3D 중심 좌표가 오차 범위 내로 계산되는지 검증합니다.

검증 2. 3D 충돌 회피 및 주행 (네비게이션)

2D 맵이 아닌 **'3D Map (Voxel Layer)'**을 써야만 안전하게 주행할 수 있는 상황을 검증합니다.

Case 4 (핵심 방어)

상단 돌출물 충돌 회피

상황 바닥 통로는 비어있으나, 선반 중간 층의 판자나 적재된 박스가 통로 쪽으로 길게 튀어나와 있음 (Overhanging obstacle).

검증 목표 바닥만 스캔하는 2D LiDAR의 한계를 극복하고, 3D Voxel Layer가 돌출물을 장애물로 맵핑하여 로봇 본체(높이 솟은 카메라 센서 등)가 부딪히지 않도록 안전한 우회 경로를 생성하는지 검증합니다.

Case 5

일반 동적/정적 장애물 우회

상황 통로 한가운데 무작위 상자가 놓여 있거나 좁은 교차로를 지나야 함.

검증 목표 RTAB-Map이 생성한 전역 지도와 Nav2의 Local Costmap이 정상 연동되어, 동적인 환경 변화에서도 충돌 없이 목표 관측 위치까지 유연하게 주행하는지 확인합니다.

검증 3. 예외 및 실패 처리 (Edge Cases)

시스템의 완성도를 높이고 최종 데모(Demo)에서 **예상치 못한 상황에 대비**하기 위한 테스트입니다.

Case 6

재고 없음 (True Negative 판정)

상황 사용자가 찾으라고 지시한 상품이 해당 선반 구역에 아예 존재하지 않음.

검증 목표 로봇이 접근 가능한 모든 후보 관측 위치를 순회하며 확인했음에도 객체가 검출되지 않으면, 제자리에서 맴돌지 않고 "상품 존재하지 않음"을 정확히 판정하여 시스템에 보고하는지 확인합니다.

Case 7

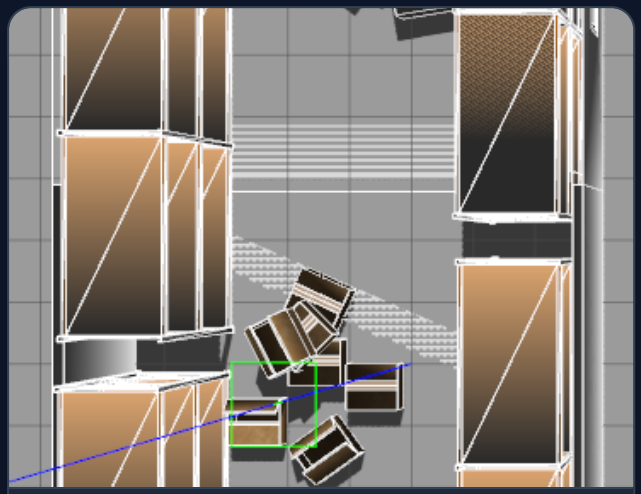
좁은 통로 진입 불가 (Planning Failure)

상황 통로가 너무 좁거나 주변에 박스가 너무 많이 쌓여 있어 로봇의 크기(Footprint) 상 계산된 관측 위치로 도달할 수 없음.

검증 목표 무리하게 선반이나 박스에 부딪히며 진입을 시도하지 않고, Nav2 Planner가 경로 생성 불가를 띄우며 안전하게 실패(Abort) 상태로 전환하는지 테스트합니다.

Gazebo 시뮬레이션 환경 구축

테스트 시나리오 검증을 위한 **3가지 커스텀 맵 디자인 및 환경 설정**



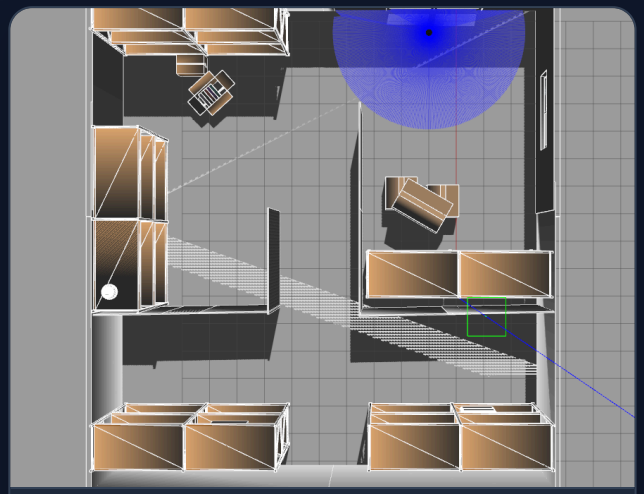
Map 1: 베이스라인 맵

(기초 인지/주행 점검)

환경 넓은 일직선 통로, 복잡한 장애물 없음

배치 선반 1층(가장 낮은 곳)에 목표 상품 배치. 통로 한가운데 가볍게 피할 수 있는 작은 상자 존재

Case 3 Case 5



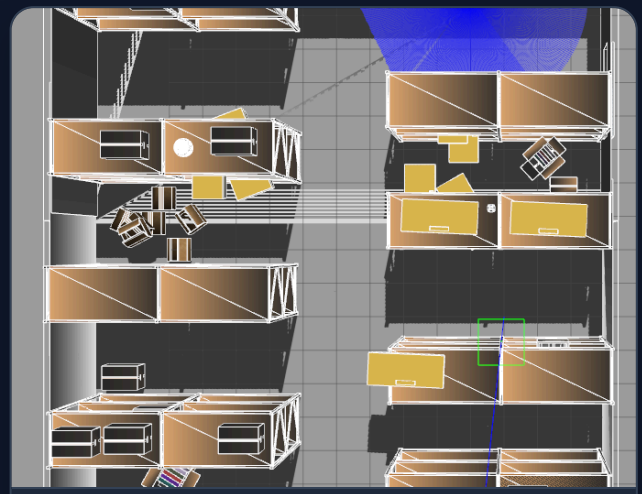
Map 2: 3D 시야/탐색 한계

(차폐 및 화각 극복)

환경 교차로가 있는 'ㅜ'자 형태의 통로

배치 선반 3층(가장 높은 곳)에 상품 배치. 특정 상품 앞 큰 박스로 정면 시야 완전 차폐. 텅 빈 선반 구역 존재

Case 1 Case 2 Case 6



Map 3: 3D 주행 극한 테스트

(안전 및 예외 상황)

환경 폭이 좁고 실제 재고창고와 비슷하게 선반이 촘촘하게 배치돼있는 구조

배치 선반 2층 높이에서 통로 쪽으로 길게 튀어나온 구조물(돌출 장애물). 박스 무더기로 막힌 통로

Case 4 Case 7

어려웠던 점 및 구현 한계 (1)

⚠ Gazebo 객체 모델 한계

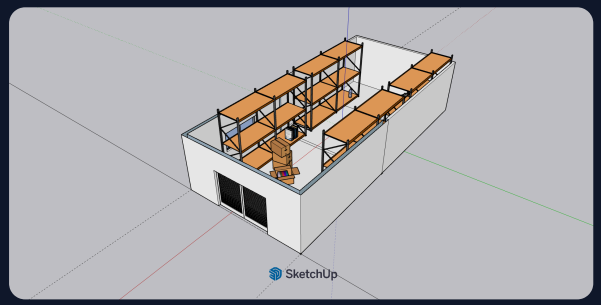
Gazebo 내에서 Map을 제작할 때 극심한 렉과 프로그램 멈춤(Crash) 현상이 빈번하게 발생하여 진행에 큰 어려움이 있었습니다.

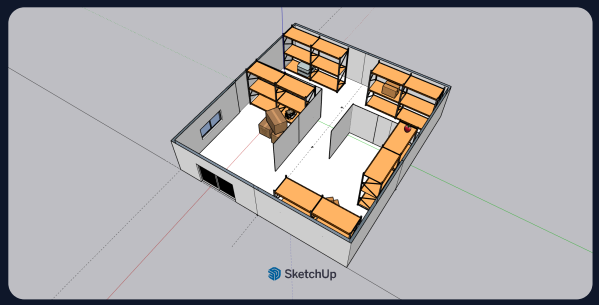
→ **대안:** 구글 스케치업(SketchUp)으로 맵을 모델링한 후, Gazebo Model로 불러와 World를 구축하는 방식으로 해결했습니다.

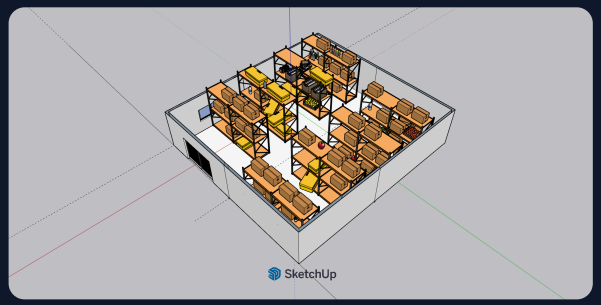
인지 성능 한계

시뮬레이션 내 객체 모델의 **텍스처가 실제 사물에 비해 지나치게 단순**하여, 현실 환경과 가상 환경 간의 차이(Domain Gap)가 발생했습니다.

→ **결과:** YOLO 네트워크의 현실 세계 탐지 성능에 비해, 가상 환경에서의 인식률(성능) 저하 한계가 일부 존재했습니다.



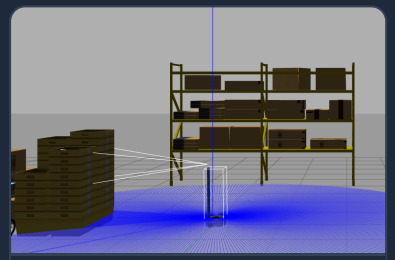




어려웠던 점 및 구현 한계 (2)

로봇 모델 개조

- 기본 TurtleBot3 Waffle 모델은 2D LiDAR와 RGB-D 센서가 있어 선택했으나, **카메라 위치가 너무 낮아 내 프로젝트에는 한계가 존재**한다고 판단했습니다.
- 실제 재고관리 로봇 사례처럼 **로봇 본체 위에 기둥을 세우고 카메라를 위로 올려서**, 선반 2~3층 물체를 탐지 가능하도록 확장 개조를 시도했습니다.
- 이 과정에서 센서 프레임 및 TF 설정 문제, 구조물 간 모델 충돌(Collision)이 발생해 조율에 어려움이 있었습니다.



Gazebo 내 로봇 모델 개조



참고한 실제 재고관리 로봇

Navigation 안정성

- **Costmap 생성 불안정:** 복잡한 선반 구조 내부에서는 Costmap 갱신이 불안정해지는 문제가 있었습니다.
- **충돌 모델 영향:** Gazebo Collision 모델의 영향으로 눈에 보이지 않는 벽에 걸려 로봇이 예기치 않게 정지하는 현상이 있었습니다.
- **재현성 문제:** 장애물이 밀집된 일부 시나리오에서는 데모의 일관된 재현성(Reproducibility)을 확보하는 데 한계가 있었습니다.

감사합니다.